Introduction to Neural Networks

Previous session

In the previous session, you learned about:

- Biological inspiration for Neural Networks,
- Mathematical representation of Neuron,
- Perceptron and basic components of perceptron,
- Perceptron Learning Model,
- Numerical problem.

Contents

- Types of Perceptron,
- Single Layer Perceptron numerical problem (OR Gate)
- Limitations of Single Layer Perceptron
- Introduction to Google Colab and the implementation of single layer perceptron

Types of Perceptron

1. Single Layer Perceptron model: One of the simplest forms of Artificial Neural Networks (ANN), limited to ***learning linearly separable patterns***.

The **architecture** consists of **only an input layer and an output layer** (with no hidden layers).

The **activation function** used is typically a **step or threshold function**.

It is more suitable for **binary classification (1 or 0).**

The model **uses the perceptron learning rule and does not employ backpropagation.**

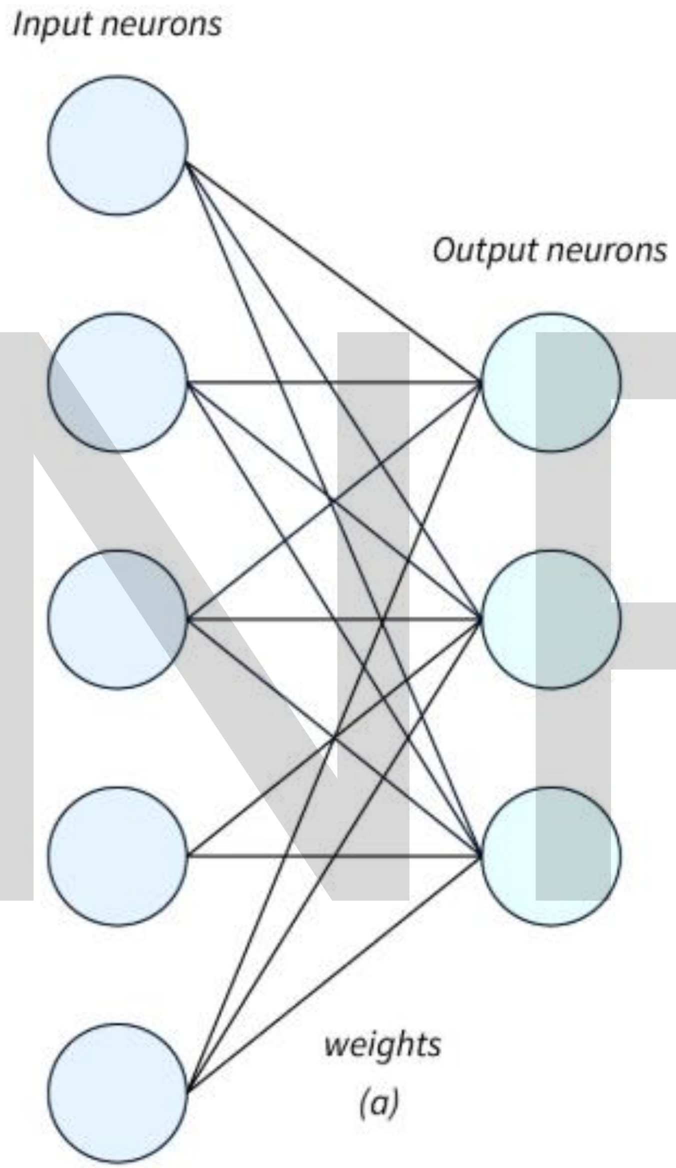
Update the weights and bias

$$w_{new} \leftarrow w_{old} + \Delta w$$

$$\Delta w = \eta(target - computed)x_i$$

$$bias_{new} \leftarrow bias_{old} + \eta(target - computed)$$

Cannot handle non-linearly separable data



Single Layer Perceptron

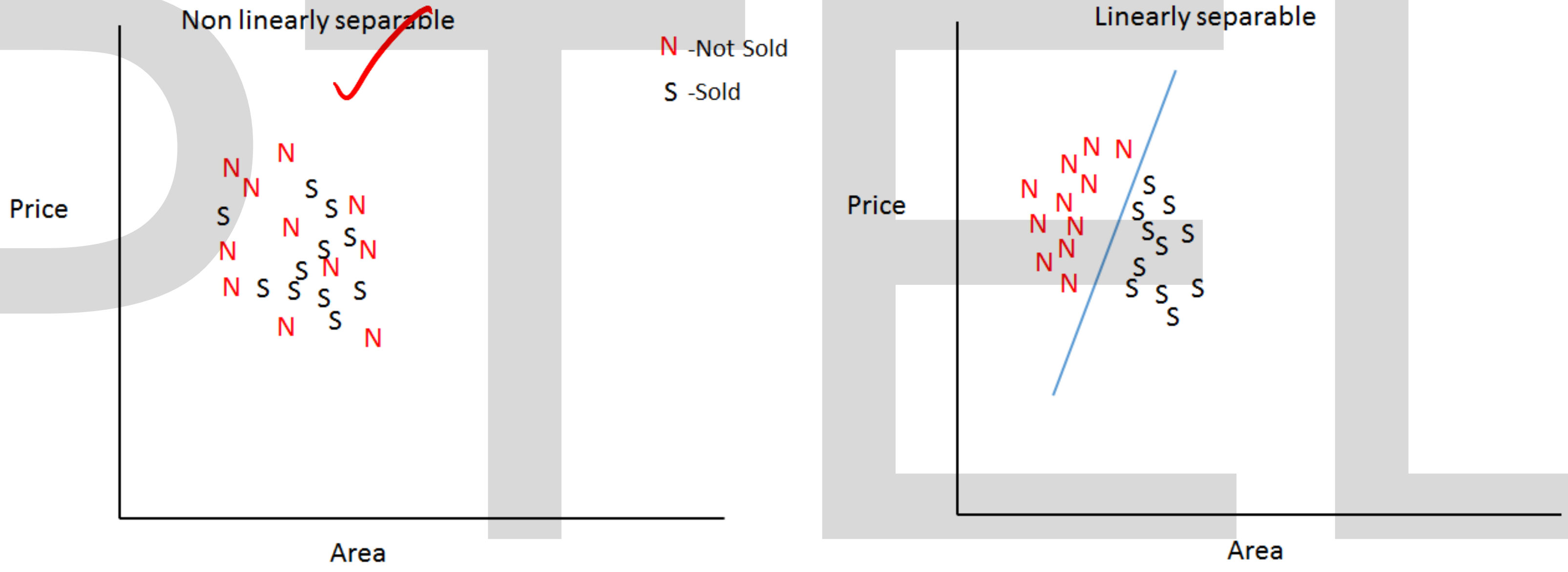


Image Source: Machine Learning Quick Reference book

Cannot handle non-linearly separable data

Types of Perceptron

2. Multi Layer Perceptron model: Capable of learning ***both linearly and non-linearly separable patterns.***

The **architecture** consists of an **input layer, one or more hidden layers, and an output layer.**

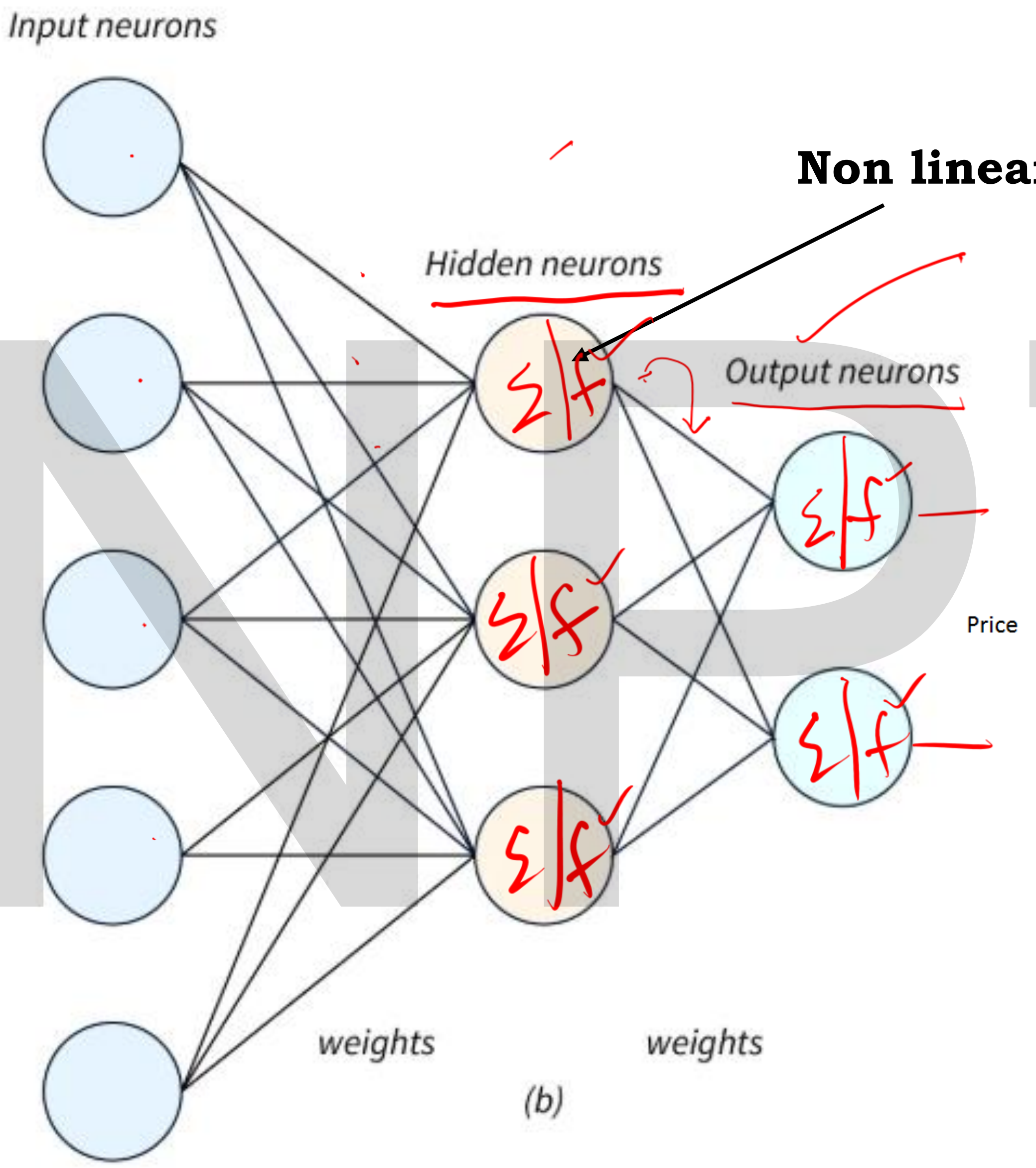
The **activation functions** used are typically **non-linear, such as tanh, ReLU, Leaky ReLU, etc.**

It is **suitable for both classification and regression tasks.**

The MLP is **trained using the backpropagation algorithm with gradient descent or its variants.**

It is mainly used for solving complex real-world problems involving non-linear decision boundaries.

Non linear activation functions



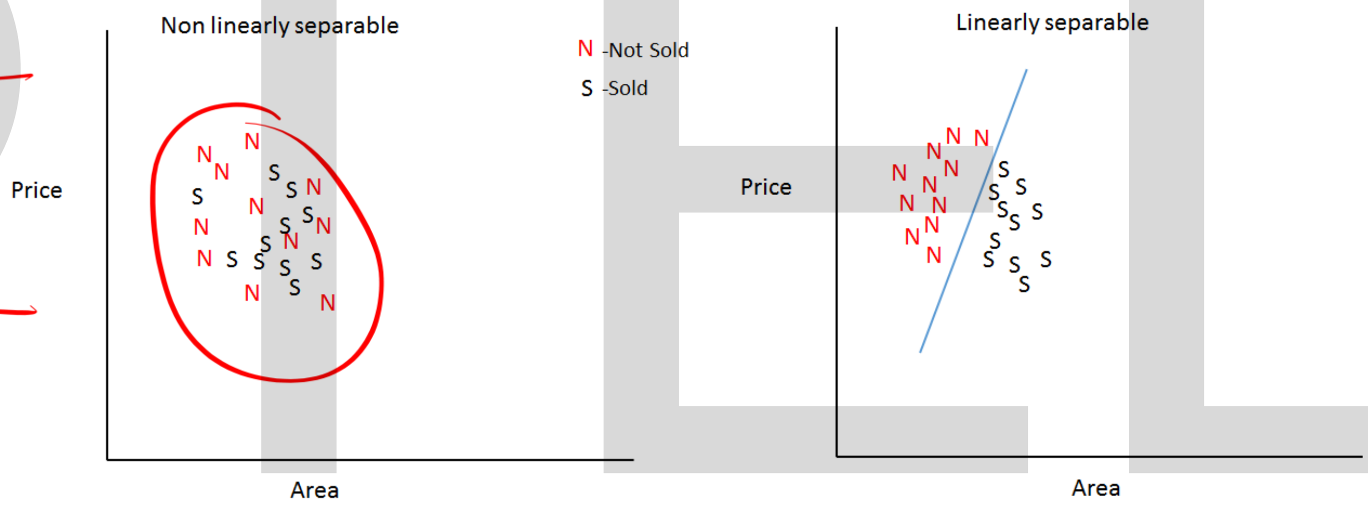
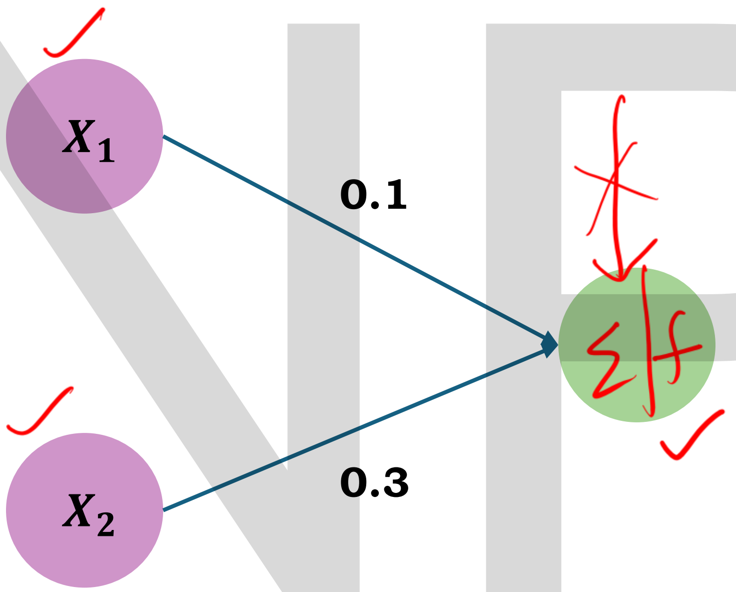


Image Source: Machine Learning Quick Reference book

Multi-Layered Perceptron

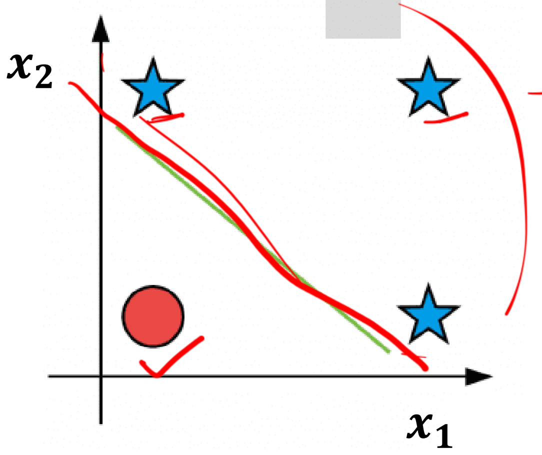
Numerical Example 2: Single Layer Perceptron



x1	x2	Target
0	0	0
0	1	1
1	0	1
1	1	1

OR

> 1



$\eta = 0.2\ (learning\ rate)$

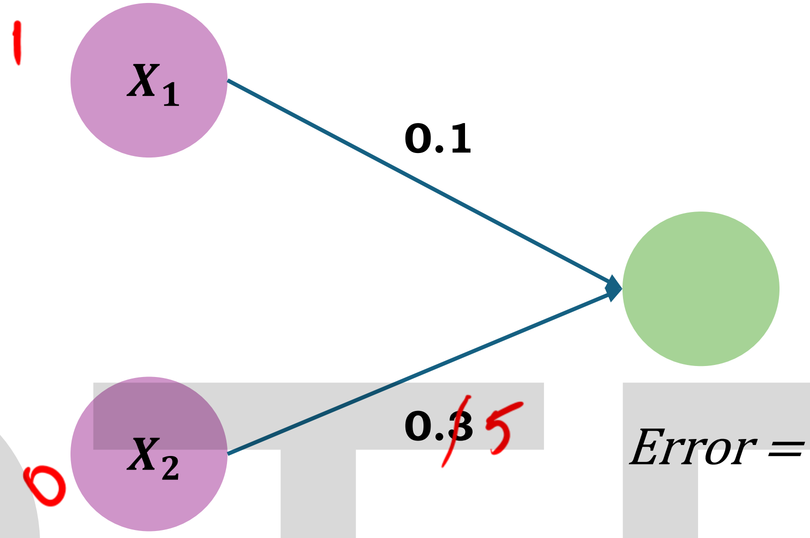
$\theta = 0.5\ (Threshold)$

$\geq 0.5 = 1$

$< 0.5 : 0$

‘OR’ Problem

x1	x2	Target
0	0	0
0	1	1
1	0	1 ✓
1	1	1



$\eta = 0.2$ (*learning rate*)

$\theta = 0.5$ (*Threshold*)

Error = Target – Computed

$\Delta w_i = \eta * error * x_i$

Epoch 1

x1	x2	W11	W21	Computed	Target	Error	ΔW_{11}	ΔW_{21}
0	0	0.1	0.3	0	0	0	0	0
0	1	0.1	0.3	0	1	1	0	0.2
1	0	0.1	0.5	0	1	1	0.2	0
1	1	0.3	0.5	1	1	0	0	0

$z = 0.1 + 0$

$z = 0.1 <$

$\hat{y} = 0 \quad t = 1$

$e = t - c = 1 - 0 = 1$

$\Delta w_1 = 0.2(1)(1)$

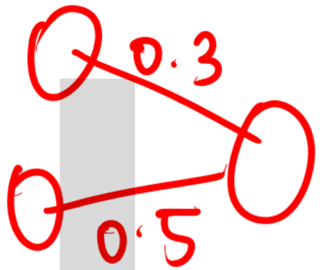
$\Delta w_1 = 0.2$

$w_1 = 0.1 + 0.2$

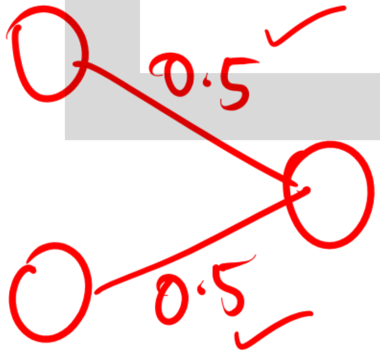
$\boxed{w_1 = 0.3}$

x1	x2	W11	W21	Computed	Target	Error	ΔW_{11}	ΔW_{21}
0	0	0.3	0.5	0	0	0	0	0
0	1	0.3	0.5	1	1	0	0	0
1	0	0.3	0.5	0	1	1	0.2	0
1	1	0.5	0.5	1	1	0	0	0

0 0
0 1
1 0
1 1



x1	x2	W11	W21	Computed	Target	Error	ΔW_{11}	ΔW_{21}
0	0	0.5	0.5	0	0	0	0	0
0	1	0.5	0.5	1	1	0	0	0
1	0	0.5	0.5	1	1	0	0	0
1	1	0.5	0.5	1	1	0	0	0



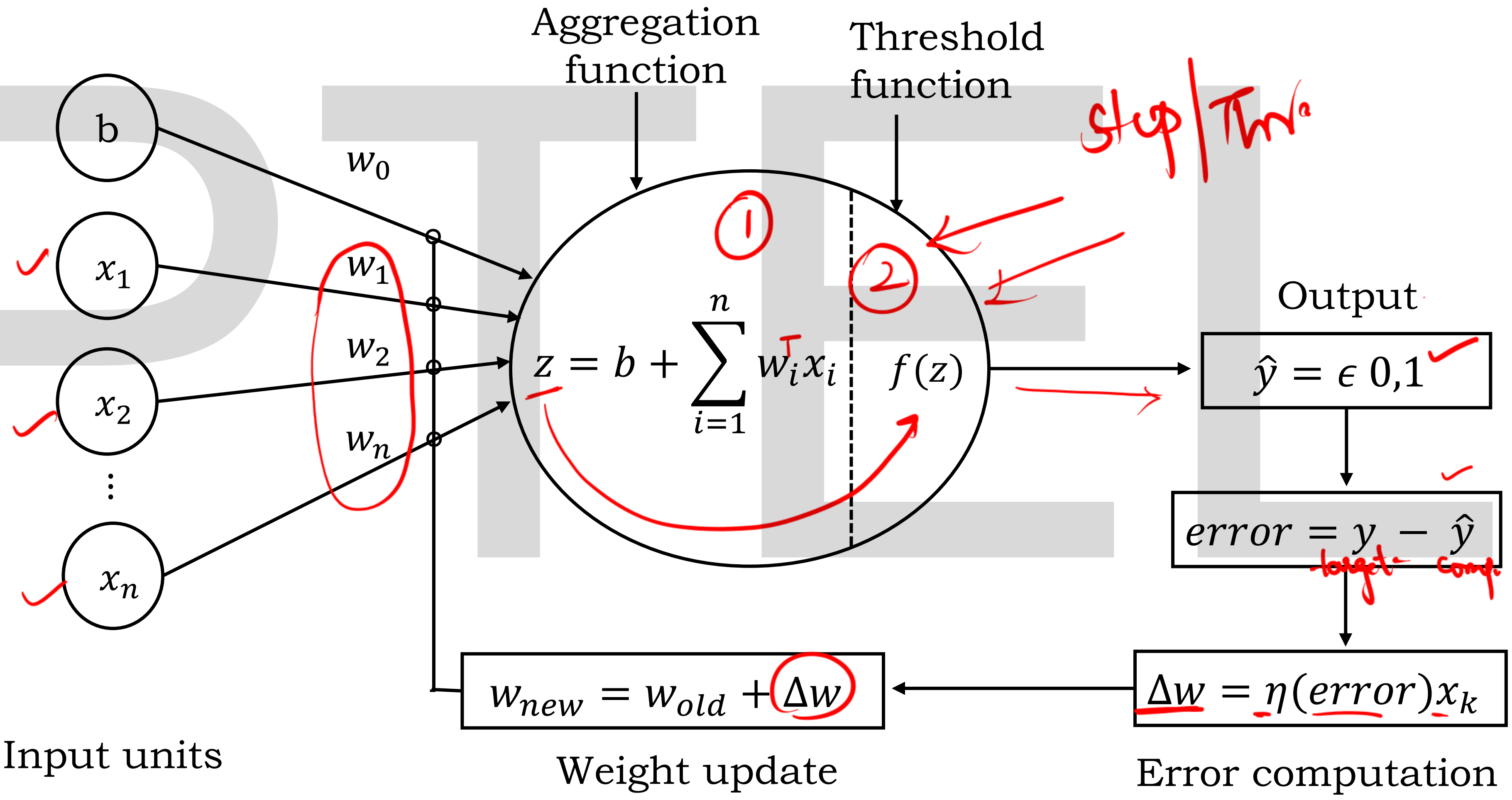
Limitations of Single layer perceptron

The following are the limitation of a single layer perceptron:

- A single-layer perceptron **can only classify data that is linearly separable.** This means it can only find a linear decision boundary (a straight line or hyperplane) to separate different classes. **It cannot solve problems where the data cannot be separated by a single line**, such as the XOR problem.
- The original single-layer perceptron is **designed for binary classification problems**, meaning it can only classify data into two distinct categories (e.g., 0 or 1). It is **not inherently capable of handling multi-class classification problems** without modifications or extensions.
- When training starts, these weights are given some **initial random values**. If these starting values are **too large, too small, or poorly chosen**, then:
  - The perceptron may take **longer to learn**, or
  - It might **get stuck** in a wrong decision boundary and never reach the correct one.

The performance of the single-layer perceptron **can be sensitive to the initial values of its weights.**

Summary



Aggregation function
Threshold function
b
x_1
x_2
x_n
w_0
w_1
w_2
w_n
$z = b + \sum_{i=1}^{n} w_i x_i$
$f(z)$
Output
$\hat{y} = \epsilon\ 0,1$
$error = y - \hat{y}$
$\Delta w = \eta(error)x_k$
$w_{new} = w_{old} + \Delta w$
Input units
Weight update
Error computation

Introduction to Google Colab and Implementation of a Single-Layer Perceptron